Quiz 3 — Practice Problems

References: Tannehill, J. C., Anderson, D. A., & Pletcher, R. H. (1997). *Computational fluid mechanics and heat transfer* (2nd ed.). Taylor & Francis [**TAP**]; Toro, E. F. (2009). *Riemann solvers and numerical methods for fluid dynamics* (3rd ed.). Springer [**T**]; Zingale, M. (2022). *Introduction to computational astrophysical hydrodynamics* [**Z**].
Course repository: https://github.com/wbandabarragan/computational-fluid-dynamics

Scope: Lectures 10 to 16, that is Unit 3 (L10–L12) and Unit 4 (L13–L16).

How to work through it

- **The quiz is written, and taken without a computer, a notebook or a phone.** You should be able to write the governing equations and the standard difference formulae, and to recognise a scheme when you see its stencil.
- **Practise the algebra by hand:** expand a Taylor series, build a finite difference from it, establish the truncation error and its order, and carry a von Neumann analysis all the way to a condition on stability.
- **Practise writing pseudocode:** For every scheme, you should be able to write the pseudocode and draw the workflow diagram: the grid set-up, the initial and boundary conditions, the time step, the loops and solution update, and the error estimations.
- Attempt each problem yourself before looking anything up. Submission is not needed. This sheet is practice for Quiz 3.

Part A — What to study

Unit 3 — Foundations of CFD and discretisation

L10. Conservation of mass, momentum and energy; the continuity equation; the Euler equations and the equation of state; the viscous stress tensor; the Navier–Stokes equations; conservative and primitive variables. Types of PDEs.

L11. Grids, stencils and nodes; finite differences from Taylor series: forward, backward and central, and the one-sided 3-point formulae; truncation error, order of accuracy, consistency; round-off against discretisation error; explicit against implicit methods for the heat equation.

L12. Stability, convergence and the Lax equivalence theorem; the von Neumann (Fourier) method and the amplification factor G; the CFL condition; the matrix method; Richardson's method and why it is unconditionally unstable; DuFort–Frankel and its consistency caveat.

Unit 4 — Advection, nonlinearity and two dimensions

L13. The one-way wave equation and its characteristics; the exact solution; the CFL number; the first-order upwind scheme and its numerical diffusion; why FTCS fails; the CFL condition and the domain of dependence; nonlinear advection, steepening.

L14. The modified equation; even derivatives give dissipation and odd derivatives give dispersion; amplitude and phase error from G; dispersion relations; Lax and Lax–Wendroff methods.

L15. The inviscid Burgers equation; shock formation; the Rankine–Hugoniot condition; conservative against non-conservative form, and why the non-conservative one gives the wrong shock speed; the flux form of the update.

L16. Finite differences in 2D; the 2D heat equation and stability condition; 2D advection and stability condition; cross derivatives and the 2D Lax–Wendroff method.

Part B — Notebooks to re-run

Run these notebooks again, change the ICs, time steps, resolutions, etc., to understand the code:

`nb05_discretisation_heat_equation.ipynb` (L11)
`nb06_richardson_dufort_frankel.ipynb` (L12)
`nb07_linear_nonlinear_advection.ipynb` (L13)
`nb08_dissipation_dispersion.ipynb` (L14)
`nb09_burgers_shocks_conservation.ipynb` (L15)
`nb10_2d_diffusion.ipynb` and `nb11_2d_advection.ipynb` (L16)

Part C — Relevant problems from the textbooks

C.1 Taylor series and finite differences (L11)

1. **TAP 3.2** — three formulas compared.
2. **TAP 3.1** — third-derivative difference.
3. **TAP 3.11** — non-uniform mesh.
4. **TAP 3.14** — Crank–Nicolson error.

C.2 Stability and the von Neumann method (L12, L13)

1. **TAP 3.24** — amplification factor, substitution.
2. **TAP 3.28** — implicit heat scheme.
3. **TAP 3.29** — implicit wave scheme.
4. **TAP 3.31** — equation with source.

C.3 Modified equation, dissipation and dispersion (L14)

1. **TAP 4.7** — MacCormack, amplitude and phase.
2. **TAP 4.3** — implicit Euler scheme.
3. **TAP 4.4** — leap frog scheme.
4. **TAP 4.8** — Lax, two routes.

C.4 Advection, Burgers and conservation (L13, L15)

1. **TAP 4.53** — an expansion fan.
2. **TAP 3.5** — conservative form, nonlinear.
3. **TAP 4.57** — a double shock.
4. **TAP 4.62** — monotone schemes.

C.5 Numerical fluxes and classic schemes (L13, L15), from Toro Chaps. 5 and 13

1. **T 5.3.6** — Warming–Beam flux.
2. **T 5.3.7** — negative speed case.
3. **T 5.3.1** — central flux, instability.
4. **T 13.2.2** — Lax–Wendroff is unique.

C.6 Two dimensions (L16)

1. **TAP 3.10** — diagonal 5-point Laplacian.
2. **TAP 3.3** — conservation property.
3. **TAP 3.4** — a second discretisation.
4. **TAP 4.21** — windward in 2D.

Part D — Pseudocode and workflow diagrams

Write pseudocode for each of the problems below, and beside it a workflow diagram with appropriate symbols. Mark the data I/O, the operations, the decisions and the loops, and name every function with its inputs and its outputs.

1. **Diffusion in 2D:** A square metal plate is held at a fixed temperature on its four edges and heated in the middle at $t = 0$. Write the pseudocode of a python program that builds the 2D grid, sets the initial and the boundary conditions, fixes the time step from the stability condition, marches the temperature with the explicit scheme, and saves a labelled 2D map of $T(x, y)$ every N steps.

2. **A solver for linear advection:** The linear advection equation is to be solved on a periodic domain of length L, discretised with N cells, starting from a smooth initial profile. Write the pseudocode of a python program that builds the grid, sets the initial and the boundary conditions, fixes the time step from a given CFL number, marches the solution to a final time $t_{\max}$ with the first-order upwind scheme, and plots the result against the exact solution.

3. **Conservative and non-conservative Burgers:** A right-propagating shock is set up on a domain of N cells as an initial jump from u_l to u_r. Write the pseudocode of a python program that marches this initial condition with both forms of the inviscid Burgers equation, computes $\int u\,dx$ at every time step, locates the front at each time, and reports the shock speed measured in each of the two runs together with the value given by the Rankine–Hugoniot condition.

If you want to solve more: TAP 3.12, 3.25, 3.26 and 3.30; TAP 4.2, 4.5, 4.12, 4.17, 4.20 and 4.49; TAP 4.27, on the ADI method; **T** 5.3.4, 5.3.5 and 13.2.1, and Toro's Tests 1 to 3 of Sect. 5.5; and **Z** 4.1 to 4.7, 6.1, 6.3 and 10.1.